

Hands-on 4: Difference Between JPA, Hibernate and Spring Data JPA

Introduction

Modern Java applications frequently interact with relational databases. Writing SQL queries manually for every database operation can be time-consuming and difficult to maintain. ORM (Object Relational Mapping) frameworks simplify this process by mapping Java objects to database tables.

Three commonly used technologies in Java persistence are:

- Java Persistence API (JPA)
- Hibernate
- Spring Data JPA

Although they are closely related, each serves a different purpose.

What is JPA?

JPA (Java Persistence API) is a Java specification that defines a standard way to map Java objects to relational database tables.

Features

- Standard specification for ORM
- Provides annotations such as:
 - @Entity
 - @Table
 - @Id
 - @Column
- Defines the EntityManager API
- Database independent
- Does not provide its own implementation

Advantages

- Standard API for persistence
- Portable across different ORM providers
- Easy to switch implementations

Limitation

JPA is only a specification. It cannot perform database operations by itself and requires an implementation such as Hibernate.

What is Hibernate?

Hibernate is an open-source ORM framework and one of the most popular implementations of JPA.

It maps Java objects to database tables and automatically generates SQL statements.

Features

- Implements JPA
- Generates SQL automatically
- Supports CRUD operations
- Handles transactions
- Provides caching
- Supports HQL (Hibernate Query Language)

Advantages

- Reduces JDBC code
- Database independent
- Automatic table mapping
- Better performance through caching

What is Spring Data JPA?

Spring Data JPA is a Spring Framework module built on top of JPA.

It further simplifies database programming by eliminating boilerplate code and providing ready-made repository interfaces.

Instead of writing DAO classes manually, developers only create repository interfaces.

Features

- Built on top of JPA
- Uses Hibernate internally (by default)
- Provides JpaRepository
- Automatic query generation
- Built-in CRUD methods
- Transaction management using Spring

Advantages

- Less code
- Faster development
- Easy integration with Spring Boot
- No need to write basic SQL queries

Hibernate Example

```
Session session = factory.openSession();
```

```
Transaction transaction = session.beginTransaction();
```

```
session.save(employee);
```

```
transaction.commit();
```

```
session.close();
```

In Hibernate, developers manage sessions and transactions manually.

Spring Data JPA Example

```
@Repository
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
```

```
}
```

```
@Autowired
```

```
private EmployeeRepository employeeRepository;
```

```
@Transactional
```

```
public void addEmployee(Employee employee) {
```

```
    employeeRepository.save(employee);
```

```
}
```

Spring Data JPA automatically manages database operations with very little code.

Advantages of Spring Data JPA over Hibernate

- Eliminates boilerplate code
- Automatic CRUD methods
- Repository pattern
- Automatic query generation
- Easy transaction management
- Better integration with Spring Boot
- Faster application development

Conclusion

JPA is a specification that defines standards for object-relational mapping. Hibernate is an ORM framework that implements the JPA specification and provides database interaction capabilities. Spring Data JPA is a higher-level abstraction built on top of JPA that simplifies database programming by providing repository interfaces and automatic CRUD operations.

Therefore, Spring Data JPA is widely used in modern Spring Boot applications because it reduces development effort while maintaining the flexibility and power of Hibernate.